

IoT-Based Patient Monitoring System

ESP32 · FreeRTOS · MQTT · Adafruit IO Dashboard

Author: KSVS Sobhita

Platform: Wokwi Simulator + Adafruit IO

Microcontroller: ESP32 (Dual-Core, 240 MHz)

1. INTRODUCTION

Modern hospital environments require continuous, real-time monitoring of patient vitals alongside environmental parameters to ensure patient safety and enable timely clinical interventions. This report describes the design and full implementation of an IoT-based patient monitoring system built around the Espressif ESP32 dual-core microcontroller, simulated in the Wokwi hardware emulation platform and connected to the Adafruit IO cloud via the MQTT protocol.

The system monitors six parameters across two categories:

Vital Signs — heart rate (HR), blood oxygen saturation (SpO₂), and body temperature.

Environmental Parameters — ambient room temperature, Air Quality Index (AQI), and room oxygen concentration.

In addition to passive monitoring, the system supports three channels of remote actuation commanded from the Adafruit IO dashboard: intravenous (IV) dosage level, bed elevation angle, and telemetry publish rate. Local fault injection via DIP switches allows controlled simulation of pathological states (fever, tachycardia, hypoxia) and network failures for validation and demonstration purposes.

A 128×64 OLED display provides a real-time local dashboard, and five buzzer/LED outputs generate audible and visual alerts when thresholds are breached. An offline failsafe

mechanism ensures continuous local monitoring even when cloud connectivity is unavailable.

2. SYSTEM ARCHITECTURE

2.1 Circuit Overview

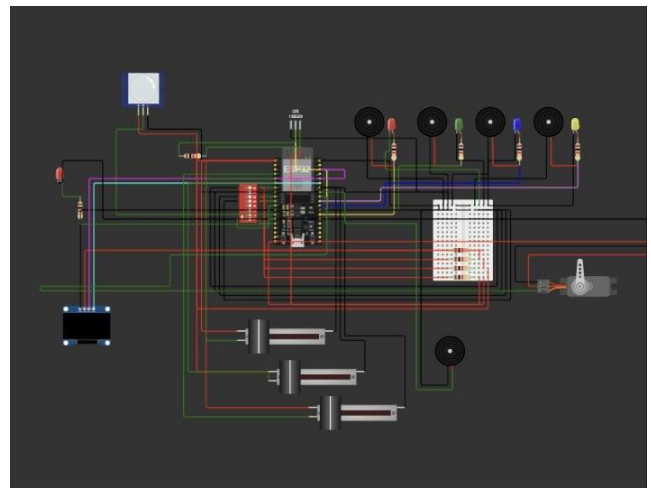


Figure 1: Wokwi circuit schematic. The ESP32 (centre) interfaces with the DS18B20 temperature sensor (top), OLED display (left), PIR motion sensor, DIP switch bank, two potentiometers, four buzzers, LEDs, and the bed elevation servo (right).

The circuit is divided into four functional zones. The *sensor zone* contains the DS18B20 OneWire temperature probe, the PIR motion detector, and two rotary potentiometers that simulate room O₂ and AQI levels. The *actuator zone* includes four condition-specific buzzers, an offline failsafe buzzer, an IV high-dosage warning LED, and a servo motor representing the bed elevation mechanism. The *human interface zone* contains the SSD1306 OLED display connected via I²C. The *fault in-*

jection zone comprises five DIP switches with internal pull-up resistors that map to specific failure or alert conditions.

2.2 Hardware Pin Assignments

Component	Interface	GPIO
DS18B20 Temperature	1-Wire	14
PIR Motion Detector	Digital In	12
Room O ₂ Potentiometer	ADC In	35
AQI Potentiometer	ADC In	39
OLED Display SSD1306	I ² C	21 / 22
	SDA/SCL	
Bed Elevation Servo	PWM Out	18
IV Dosage Warning LED	Digital Out	27
Buzzer 1 – Body Temp	Digital Out	4
Buzzer 2 – HR/BP	Digital Out	16
Buzzer 3 – SpO ₂ /Resp	Digital Out	17
Buzzer 4 – Room Temp	Digital Out	5
Offline Failsafe Buzzer	Digital Out	19
Kill Switch (DIP 1)	Digital Pull-up	13
Fever Sim (DIP 8)	Digital Pull-up	32
Tachycardia Sim (DIP 7)	Digital Pull-up	33
Hypoxia Sim (DIP 6)	Digital Pull-up	25
Motion Bypass (DIP 5)	Digital Pull-up	26

2.3 Software Architecture

The software follows a layered producer–consumer architecture distributed across the two ESP32 cores. Sensor acquisition tasks run on Core 1, isolated from the network and display logic running on Core 0. Inter-task communication uses FreeRTOS single-slot queues, which guarantee that the consumer always processes the most recent reading without blocking sensor acquisition.

3. FREERTOS TASK STRUCTURE

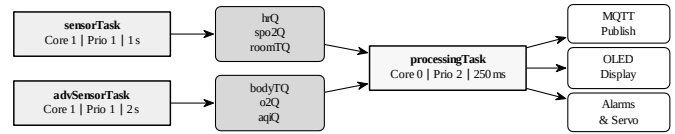


Figure 2: FreeRTOS task graph. Sensor tasks (Core 1) produce data into queues; the processing task (Core 0) consumes all queues and drives outputs.

3.1 sensorTask — Core 1, Priority 1, 1000 ms

This task handles the DS18B20 OneWire sensor and simulated vital sign generation. It calls `sensors.requestTemperatures()` each cycle and retrieves the result via `getTempCByIndex(0)`. If the sensor returns an invalid value ($\leq -100^\circ\text{C}$ or exactly 0°C), a plausible room temperature near 24°C is synthesised using `random()`.

Heart rate and SpO₂ are generated with `random()` within physiologically appropriate ranges, with the DIP switches shifting those ranges to simulate pathological states:

- SW-HR LOW $\Rightarrow$ HR = 110–135 BPM (tachycardia)
- SW-SPO2 LOW $\Rightarrow$ SpO₂ = 82–88% (hypoxia)

Results are written to `hrQueue`, `spo2Queue`, and `roomTempQueue` using `xQueueOverwrite()`.

3.2 advancedSensorTask — Core 1, Priority 1, 2000 ms

This task reads the two potentiometers via `analogRead()` and maps the 12-bit ADC values (0–4095) to physical units using `map()`:

- Room O₂: 0–4095 $\rightarrow$ 15–25%
- AQI: 0–4095 $\rightarrow$ 0–500

Body temperature is synthesised in the range 36.5–37.1 °C (normal) or 38.5–39.4 °C when SW_TEMP is LOW (fever simulation). Results are written to `roomO2Queue`, `aqiQueue`, and `bodyTempQueue`.

3.3 processingTask — Core 0, Priority 2, 250 ms

The highest-priority task runs on Core 0 and is responsible for all output-facing operations. Each 250 ms cycle it: reads all six queues with `xQueuePeek()`, checks connectivity and handles the kill switch, processes incoming MQTT subscriptions, evaluates alarm thresholds, refreshes the OLED, and conditionally publishes telemetry. Running on a dedicated core at the highest priority ensures that display latency and MQTT I/O never interfere with sensor timing on Core 1.

4. MQTT FEED MAPPING

The system communicates with Adafruit IO over MQTT on port 1883. All feed topic paths follow the format `Rushikesh23/feeds/<name>`. Eight feeds are published (outbound telemetry) and three are subscribed (inbound remote commands).

Feed Name	Dir.	Units / Range
heart-rate-monitor	Pub	BPM (60–150)
spo2	Pub	% (82–100)
body-temp	Pub	°C (36.5–39.5)
room-temp	Pub	°C (10–50)
room-oxygen	Pub	% (15–25)
air-quality	Pub	AQI (0–500)
motion-detector	Pub	Boolean 0 / 1
flag	Pub	Status string
dosage	Sub	mg (0–100)
bed-elevation	Sub	Degrees (0–180)
publish-rate	Sub	Seconds (integer)

Published feeds are guarded with range checks (e.g., body temperature only published if $> 30.0^{\circ}\text{C}$) to suppress invalid readings during sensor initialisation. The `flag` feed carries the current system status string, enabling remote staff to observe the alarm state without consulting the physical display.

5. DASHBOARD SCREENSHOTS

Two separate Adafruit IO dashboards provide different views of the system. The **Patient Monitoring Dashboard** is intended for clinical staff and shows vital-sign gauges alongside remote control widgets. The **Facility Management View** is intended for biomedical engineering or estates staff and focuses on environmental parameters.



Figure 3: Patient Monitoring Dashboard on Adafruit IO. Gauge widgets display heart rate (84 BPM) and SpO_2 (96%). A motion indicator, system flag text block, and three slider widgets for dosage (39 mg), bed elevation (60°), and sampling rate (25 s) complete the view.

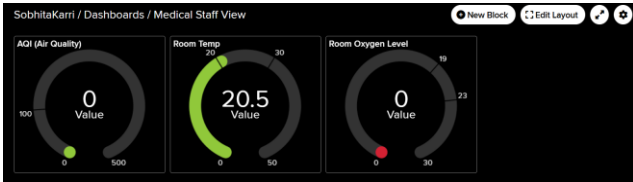


Figure 4: Facility Management View. Gauge widgets show AQI (0, safe), room temperature (46 °C, red alert – above 28 °C threshold), and room oxygen level (15%, red alert – below 19% safe lower bound).

6. REMOTE DOSAGE ADJUSTMENT LOGIC

The intravenous dosage level is controlled remotely via the dosage MQTT subscription feed. The `processingTask` calls `mqtt.readSubscription(200)` on every 250 ms cycle. When a new message arrives on the dosage feed, the library populates `dosageSub.lastread` with the ASCII payload, which is then converted to an integer:

```
currentDosage = atoi((char*)dosageSub.lastread);
```

The alarm evaluation block runs immediately afterwards. The threshold of 80 mg represents the upper safe limit for the simulated IV drip rate:

```
if (currentDosage > 80) {
    statusMsg = "WARN: High IV Dose!";
    digitalWrite(IV_LED_PIN, HIGH);
} else {
    digitalWrite(IV_LED_PIN, LOW);
}
```

When the threshold is exceeded, the IV warning LED on GPIO 27 is illuminated and the status message is updated both on the OLED and published to the `flag` feed on the next publish cycle. This dual notification (local LED + cloud flag) ensures the alert is visible regardless of whether the clinician is physically present. The Adafruit IO slider widget provides a 0–100 range, and the updated value appears on the OLED `IV:Xmg` field within one 250 ms processing cycle.

7. SERVO ANGLE MAPPING – BED ELEVATION

Bed elevation is actuated by a standard 0–180° hobby servo connected to GPIO 18 via the `ESP32Servo` library. The `bed-elevation` MQTT feed accepts an integer in the range 0–180. Reception and application follow the same pattern as dosage:

```
currentBedAngle = atoi((char*)bedSub.lastread);
bedServo.write(currentBedAngle);
```

The design deliberately uses a 1:1 mapping between the dashboard slider value and the servo angle in degrees. This eliminates any intermediate scaling function and provides full angular resolution to the remote operator. The physical interpretation of the angle in a deployment context would be:

Angle	Bed Position	Clinical Use
0°	Fully flat	Post-surgical / CPR
30–45°	Head elevated	Respiratory support
90°	Half raised	Semi-Fowler's position
180°	Maximum elevation	Acute dyspnea

The current angle is displayed on the OLED as `Bed:X deg` and is updated within one processing cycle of receiving the command. The `bedServo.write()` call is blocking for the duration of the PWM pulse but completes well within the 250 ms cycle budget.

8. DYNAMIC SAMPLING RATE IMPLEMENTATION

The telemetry publish interval is not fixed at compile time. It is stored in the variable `publishInterval` (default 20 000 ms = 20 s) and can be updated at runtime from the

Adafruit IO dashboard slider via the `publish-rate` subscription feed:

```
publishInterval =  
  atoi((char*)rateSub.lastread) * 1000UL;
```

The multiplier `1000UL` converts the slider's integer seconds value to milliseconds. The publish decision uses a non-blocking elapsed-time comparison rather than a `delay()` call:

```
if (millis() - lastPublishTime >= publishInterval) {  
  // publish all feeds  
  lastPublishTime = millis();  
}
```

This pattern is critical: because the check is embedded in the 250 ms processing loop, the OLED display and alarm evaluation continue to run at full speed regardless of the configured cloud publish interval. Clinicians can slow the rate (e.g., 60 s) to conserve Adafruit IO data-point quota during stable periods, or shorten it (e.g., 5 s) during acute monitoring. The current interval is displayed in the OLED bottom-right corner as Xs.

9. OFFLINE DETECTION & FAILSAFE MECHANISM

Network resilience is a key safety requirement: the system must continue to monitor and alarm locally even when cloud connectivity is lost. On every 250 ms processing cycle, three independent conditions are evaluated:

1. Wi-Fi link failure:

```
WiFi.status() != WL_CONNECTED
```

2. MQTT session loss:

```
!mqtt.connected()
```

3. Hardware kill switch:

```
digitalRead(KILL_SWITCH) == LOW
```

(DIP Switch 1, active-low with internal pull-up)

If any condition is true, the `isOffline` flag is set and the following failsafe responses execute in the same cycle:

- OLED status line switches to "LOGGING OFFLINE", providing clear local indication.
- The offline buzzer (GPIO 19) is pulsed HIGH for 100 ms then LOW, producing a periodic audible alert that repeats every 250 ms cycle.
- If the kill switch triggered disconnection, `mqtt.disconnect()` is called to cleanly close the TCP socket.
- All MQTT publish and subscription operations are bypassed, preventing null-pointer dereferences or blocking socket calls on a broken connection.
- Alarm threshold evaluation, OLED refresh, and local buzzer logic continue to operate normally – full graceful degradation to a standalone monitoring device.

Auto-reconnect strategy: While offline (and with the kill switch inactive), the system attempts reconnection every 5 000 ms using a timestamp comparison identical to the publish-rate mechanism. If Wi-Fi has dropped, `WiFi.reconnect()` is called first; if Wi-Fi is present but the MQTT session was lost, `mqtt.connect()` is called directly. This staged strategy avoids flooding the broker with connection attempts during transient outages and mirrors industry-standard MQTT client back-off patterns.

10. CONCLUSION

The IoT patient monitoring system demonstrates a robust, production-quality architecture suitable as a foundation for

real clinical monitoring hardware. Key design strengths include:

- **Core isolation:** Sensor acquisition on Core 1 is never affected by network or display latency on Core 0.
- **Non-blocking design:** Elapsed-time patterns replace all `delay()` calls, preserving full loop throughput.
- **Graceful degradation:** The offline failsafe ensures continuous local monitoring with audible alerts regardless of cloud connectivity.
- **Runtime configurability:** Dynamic publish rate and remote actuation (dosage, bed angle) eliminate the need for firmware reflashing during operation.
- **Fault injection:** DIP-switch simulation of pathological states enables repeatable validation without requiring actual physiological changes.

Future extensions could include persistent offline data logging to on-board flash (SPIFFS/LittleFS), TLS-encrypted MQTT communication, and integration of real pulse-oximetry and ECG sensors to replace the simulated vital sign generation.